

Memory Management and Virtual Memory



by Sunayana Bisalapur

Module 4-

Memory Management and Virtual Memory Logical Versus Physical AddressSpace, Swapping,Contiguous Memory Allocation, Paging,Structure of PageTable, Segmentation,Demand Paging,Page Replacement, Page ReplacementAlgorithms

Background

Program must be brought into memory and placed within a process for it to be run.

To improve both the utilization of the CPU and the speed of its response to users, a general-purpose computer must keep several processes in memory.

OS provides Many memory-management schemes to improve memory utilization.

Input queue or job queue – collection of processes on the disk that are waiting to be brought into memory to run the program.

OS has to perform various steps in order to execute the processes such as address and data binding.

What is Memory Management?

Memory management in OS is a technique of controlling and managing the functionality of Random access memory (primary memory). It is used for achieving better concurrency, system performance, and memory utilization.

Memory management moves processes from primary memory to secondary memory and vice versa. It also keeps track of available memory, memory allocation, and unallocated.

Logical Address and Physical Address

1

Logical Address Space

- The set of all addresses **generated by the CPU (program)**
- Also called **Virtual Address Space**
- Used by programs during execution
- Each process has its **own logical address space**
- Starts from **0**

2

Physical Address Space

- The set of all addresses **actually present in main memory (RAM)**
- Real addresses in memory hardware
- Shared by all processes
- Managed by OS

Address Binding

Address binding is the process of mapping from one address space to another address space.

The run time mapping between Virtual address and Physical Address is done by hardware device known as **MMU**.

In **memory management**, Operating System will handle the processes and moves the processes between disk and memory for execution.

It keeps the track of available and used memory.

Static and Dynamic Loading

Static Loading

Static Loading involves loading all the necessary program components into the main memory before the program's execution begins.

This means that both the executable code and data are loaded into predetermined memory locations.

This allocation is fixed and does not change during the program's execution.

While it ensures direct access to all required resources, it may lead to inefficiencies in memory usage, especially if the program doesn't utilize all the loaded components.

Dynamic Loading

Dynamic Loading takes a more flexible approach.

In this scheme, a program's components are loaded into the main memory only when they are specifically requested during execution.

This results in a more efficient use of memory resources as only the necessary components occupy space.

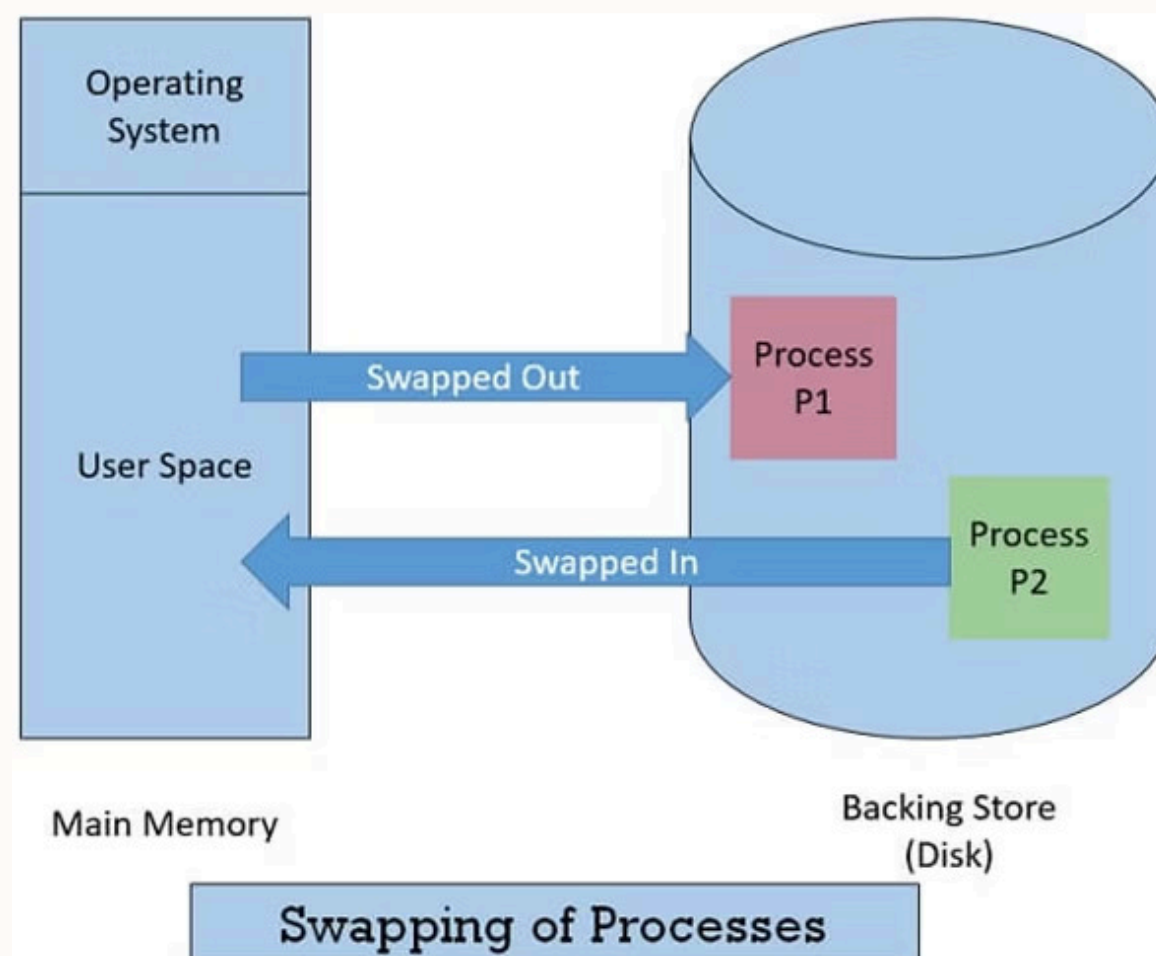
Dynamic loading is particularly advantageous for programs with extensive libraries or functionalities that may not be used in every session.

Swapping

Swapping is a mechanism in which a process can be swapped temporarily out of main memory (or move) to secondary storage (disk) and make that memory available to other processes.

A process will be moved to temporarily out of memory to a backing store and then brought back into memory for continued execution.

Swapping makes it possible for the total physical address space of all processes to exceed the real physical memory of the system, thus increasing the degree of multiprogramming in a system.



The backing store is a fast disk which must be large enough to accommodate copies of all memory images for all users.

Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory swapped

Benefits of Swapping:

- It offers a higher degree of multiprogramming.
- Allows dynamic relocation.
- It helps to get better utilization of memory

Contiguos Memory Allocation

Contiguous memory allocation is a memory management technique that involves allocating a process to the entire contiguous block of the main memory it requires to execute.

This means that the process is loaded into a single continuous chunk of memory. While it's straightforward and efficient in terms of execution, it can lead to issues with fragmentation, where smaller blocks of memory remain unused between allocated processes.

Memory Allocation

Memory allocation is the process of reserving a portion of the computer's memory for a specific application or program. It's a crucial aspect of memory management, ensuring that each running process has enough space to execute efficiently. Effective memory allocation strategies are essential for optimizing system performance.

1

First Fit

First Fit is a memory allocation algorithm that allocates the first available block of memory that is large enough to accommodate a process. It scans the memory from the beginning and selects the first block that meets the size requirements. While it is relatively simple to implement, it can lead to fragmentation over time.

Example:

Free blocks: 100, 500, 200, 300
Process = 180
Allocated to **500** (first suitable)

2

Best Fit

The Best Fit algorithm searches the entire memory space to find the smallest block that can accommodate a process. This helps in minimizing wastage of memory, as it selects the smallest available block that fits. However, it may lead to more fragmented memory spaces compared to other allocation strategies

Example:

Blocks: 100, 500, 200, 300
Process = 180
Allocated to **200** (best match)

3

Worst Fit

Worst Fit, as the name implies, allocates the largest available block of memory to a process. This approach can result in more fragmentation compared to First Fit or Best Fit strategies. However, it can be useful in scenarios where larger processes need to be accommodated.

Example:

Blocks: 100, 500, 200, 300
Process = 180
Allocated to **500** (largest)

Difference between Contiguous and Non Contiguous

Contiguous Memory Allocation	Non-contiguous Memory Allocation
Contiguous memory allocation allocates consecutive blocks of memory to a file/process.	Non-Contiguous memory allocation allocates separate blocks of memory to a file/process.
Faster in Execution.	Slower in Execution.
It is easier for the OS to control.	It is difficult for the OS to control.
Overhead is minimum as not much address translations are there while executing a process.	More Overheads are there as there are more address translations.
Internal fragmentation occurs in Contiguous memory allocation method.	External fragmentation occurs in Non-Contiguous memory allocation method.
It includes single partition allocation and multi-partition allocation.	It includes paging and segmentation.
Wastage of memory is there.	No memory wastage is there.
In contiguous memory allocation, swapped-in processes are arranged in the originally allocated space.	In non-contiguous memory allocation, swapped-in processes can be arranged in any place in the memory.

Fragmentation

Fragmentation is an unwanted memory space problem in the operating system occurred when the processes are loaded and unloaded from memory.

Internal Fragmentation

When a process with smaller amount of memory request is allocated with a larger memory block, then a free space is created in the given memory block that becomes unused. This condition is called as internal fragmentation.

Example:

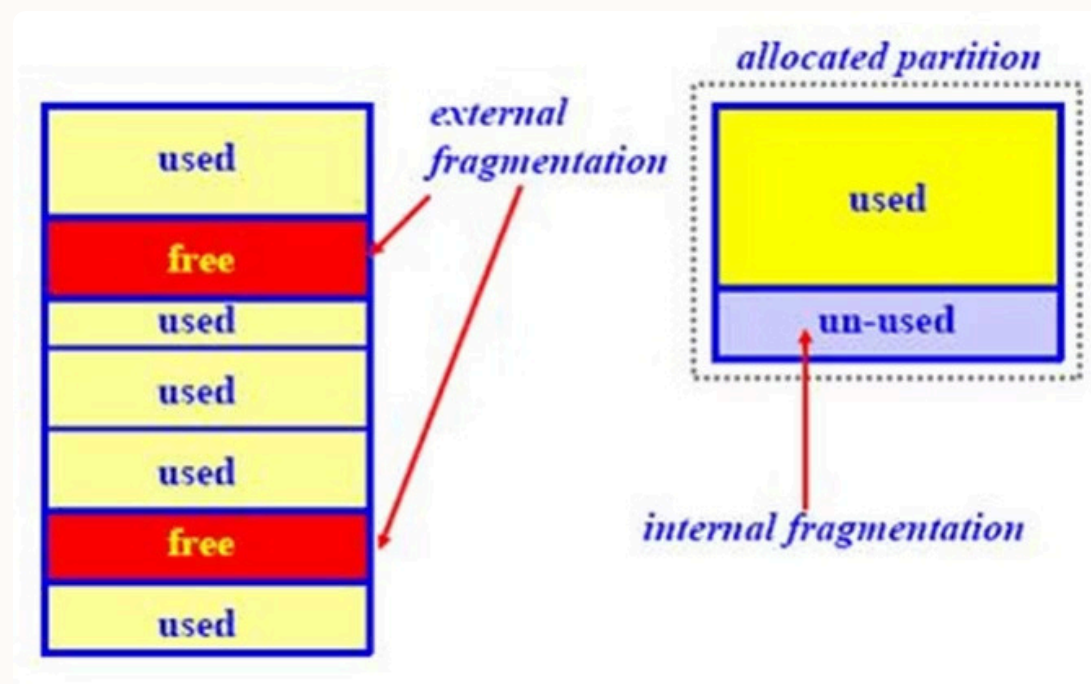
- Process needs 18 KB
- Allocated block = 20 KB
- **2 KB wasted inside** → Internal fragmentation

External Fragmentation

External fragmentation occurs when there is enough memory space to complete a memory request of some process, but it is not contiguous i.e., Free memory is **available but scattered in small pieces**

Example:

- Free spaces: 5 KB + 3 KB + 2 KB
- Need: 10 KB
- Cannot allocate (even though total = 10 KB)



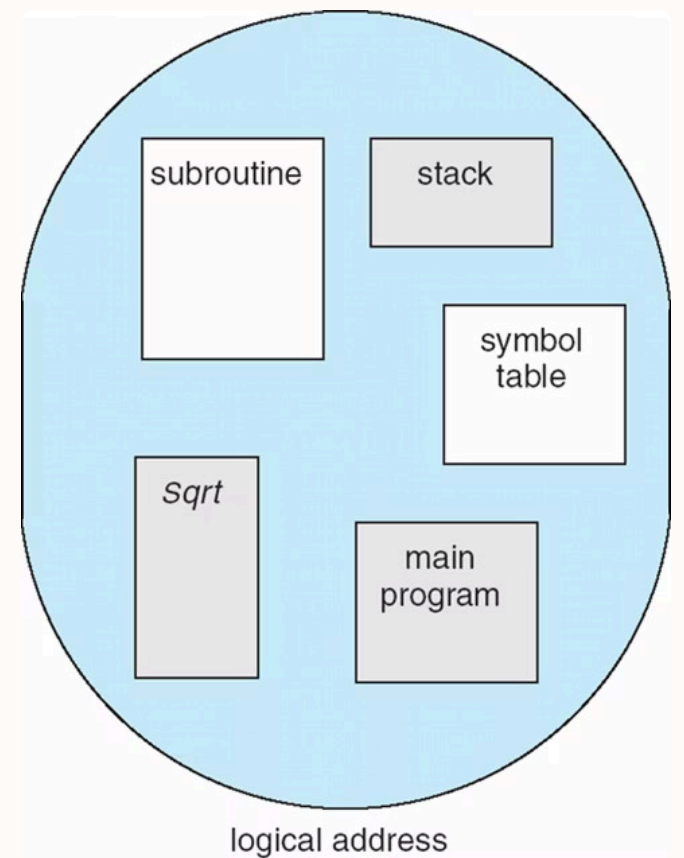
Segmentation

- Segmentation divides processes into smaller subparts known as modules. The divided segments need not be placed in contiguous memory. Since there is no contiguous memory allocation, internal fragmentation does not take place. The length of the segments of the program and memory is decided by the purpose of the segment in the user program.
- We can say that logical address space or the main memory is a collection of segments.
- Segmentation permits the physical address space of a process to be noncontiguous.
- Each segment has a name and a length. The addresses specify both the segment name and the offset within the segment.
- For simple implementation segments are numbered and are referred to by a segment number, rather than by a segment name.
- Thus, a logical address consists of a two tuple:
- **<segment-number, offset>.**

Segmentation

Compiler might create separate segments for the following:

1. The code
2. Global variables
3. The heap, from which memory is allocated
4. The stacks used by each thread
5. The standard C library



Segmentation Hardware

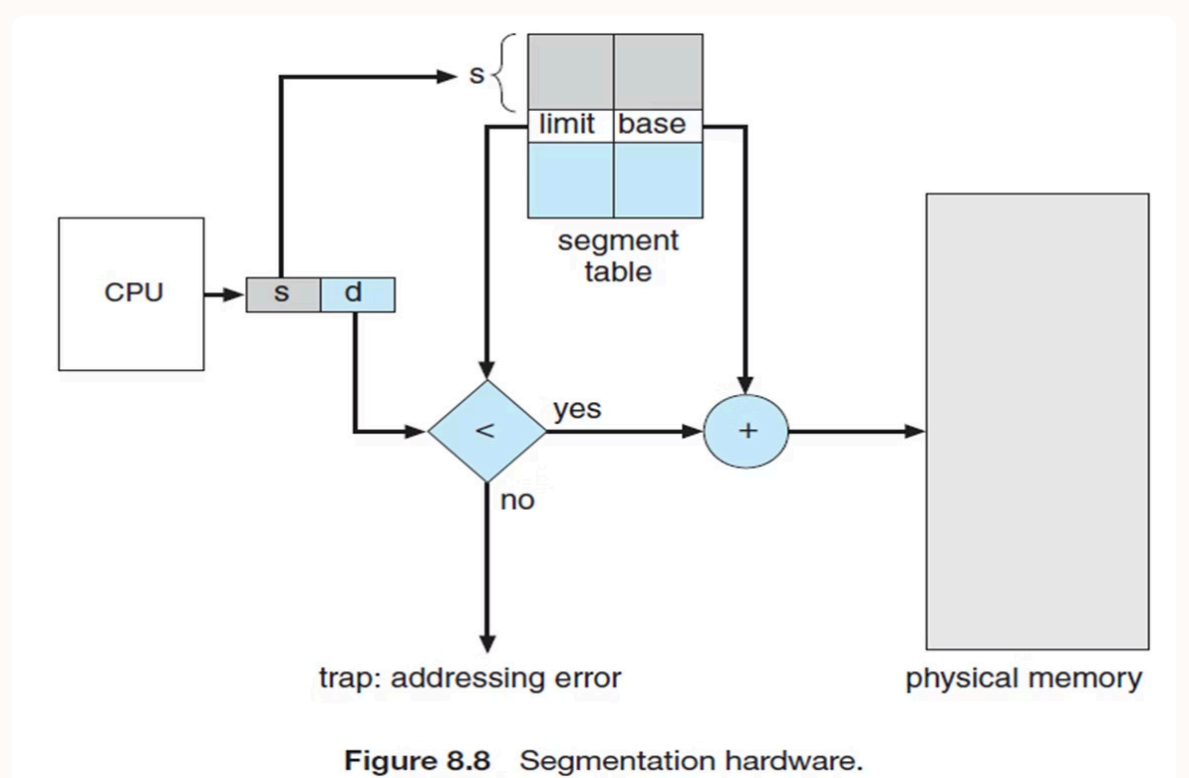
It is used to map two-dimensional user-defined addresses into one-dimensional physical address with the help of segment table.

Each entry in the segment table has a segment base and a segment limit.

The segment base contains the starting physical address where the segment resides in memory, and the segment limit specifies the length of the segment.

Address generated by the CPU is divided into:

1. Segment number (s): Number of bits required to represent the segment.
2. Segment offset (d): Number of bits required to represent the size of the segment.



Advantages of Segmentation –

- No Internal fragmentation.
- Segment Table consumes less space in comparison to Page table in paging.

Disadvantage of Segmentation –

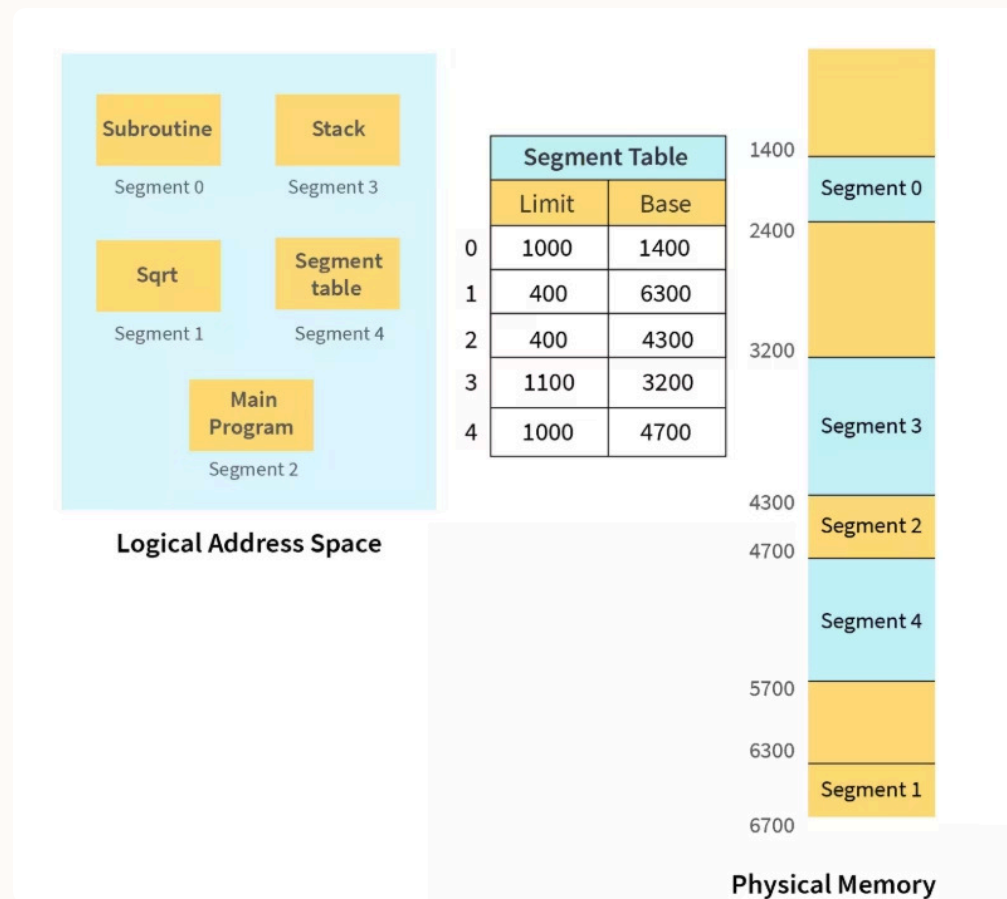
As processes are loaded and removed from the memory, the free memory space is broken into little pieces, causing External fragmentation

Example of Segmentation

Let us assume we have five segments namely: Segment-0, Segment-1, Segment-2, Segment-3, and Segment-4. Initially, before the execution of the process, all the segments of the process are stored in the physical memory space.

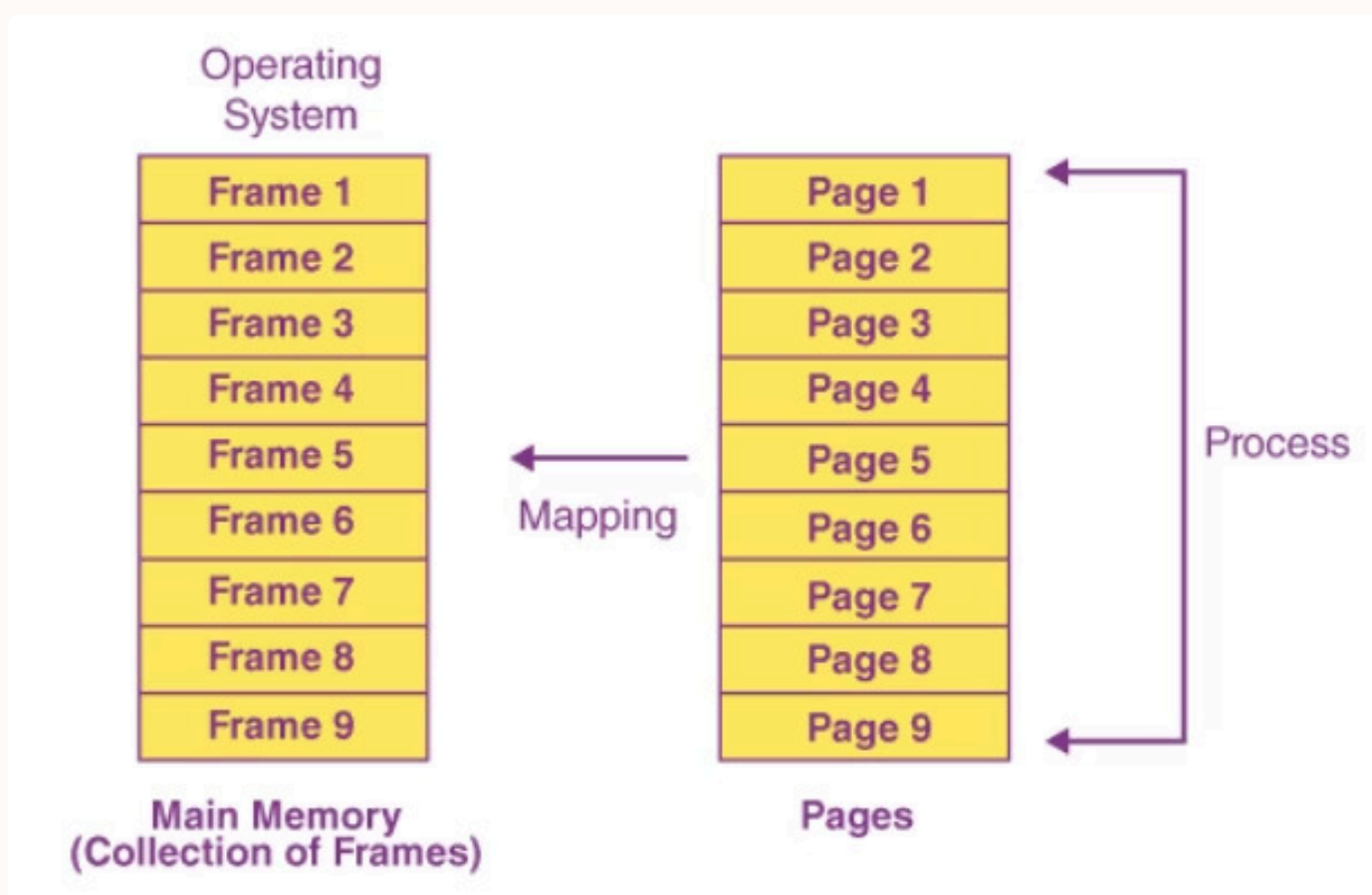
The segment table contains the beginning entry address of each segment (denoted by base). The segment table also contains the length of each of the segments (denoted by limit).

As shown in the image the base address of Segment-0 is 1400 and its length is 1000, the base address of Segment-1 is 6300 and its length is 400, the base address of Segment-2 is 4300 and its length is 400, and so on.

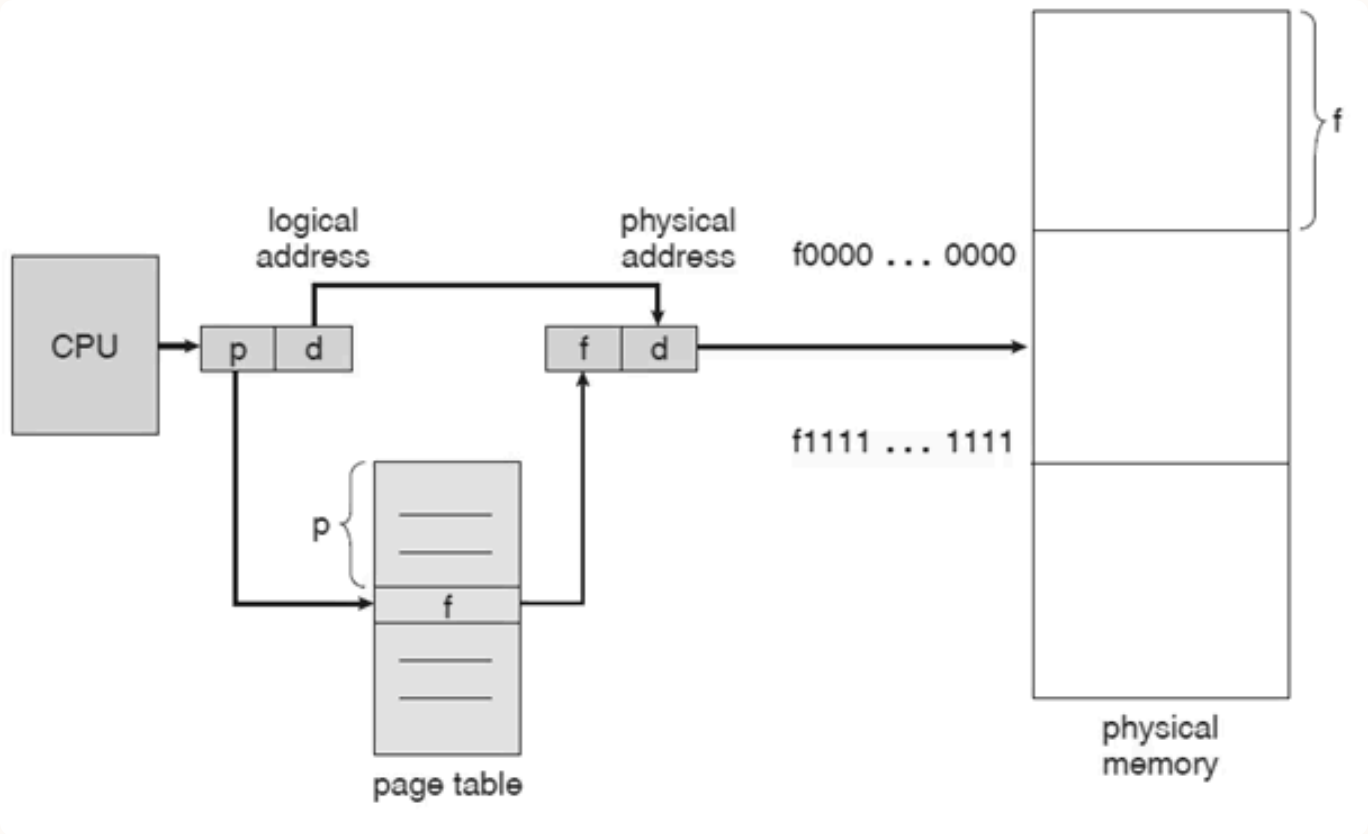


Paging

- Paging is a memory management technique in operating systems that eliminates the need for contiguous allocation of physical memory
- It breaks down processes into small, fixed-size chunks called pages (logical memory) and divides the RAM into same-sized blocks called frames (physical memory).
- This concept of Paging in OS includes dividing each process in the form of pages of equal size and also, the main memory is divided in the form of frames of fixed size.
- Now, each page of the process when retrieved into the main memory, is stored in one frame of the memory, and hence, it is also important to have the pages and frames of equal size for mapping and maximum utilization of the memory.
- Its main advantage is that the pages can be stored at different locations of the memory and not necessarily in a contiguous manner, though priority is always set to firstly find the contiguous frames for allocating the pages.



Paging Hardware



- The paging technique divides the physical memory(main memory) into fixed-size blocks that are known as Frames and also divide the logical memory(secondary memory) into blocks of the same size that are known as Pages.
- Pages of a process are brought into the main memory only when there is a requirement otherwise they reside in the secondary storage.
- One page of a process is mainly stored in one of the frames of the memory. Also, the pages can be stored at different locations of the memory but always the main priority is to find contiguous frames

Translation of Logical Address into Physical Address

- The CPU always generates a logical address.
- In order to access the main memory always a physical address is needed.

The logical address generated by CPU always consists of two parts:

- 1

Page Number(p)
- 2

Page Offset (d)

where,

Page Number is used to specify the specific page of the process from which the CPU wants to read the data. and it is also used as an index to the page table.

Page offset is mainly used to specify the specific word on the page that the CPU wants to read.

Page Table

- The Page table mainly contains the base address of each page in the Physical memory. The base address is then combined with the page offset in order to define the physical memory address which is then sent to the memory unit.
- Thus page table mainly provides the corresponding frame number (base address of the frame) where that page is stored in the main memory.

So, The physical address consists of two parts:

- 1

Page offset(d):
Indicates the specific word that has to be read from that page.
- 2

Frame Number(f):
It is used to indicate the specific frame where the required page is stored.
- 3

Formula: Physical Address = (Frame Number X Page Size) + Offset

It is important to note here that the size of the page is typically the power of 2 that varies between 512 bytes and 16 MB per page and it mainly depends on the architecture of the computer.

If the size of **logical address space** is 2 raised to the power m and **page size** is 2 raised to the power n addressing units then the high order **m-n** bits of **logical address designates the page number** and the **n** low-order **bits designate the page offset**.

| Page Number (m - n bits) | Offset (n bits) |

Example:

- Logical address space = $2^8 \rightarrow m = 8$ bits
- Page size = $2^4 \rightarrow n = 4$ bits

Logical address = 8 bits

Split:

- Page number = $8 - 4 = 4$ bits
- Offset = 4 bits

Logical Address (8 bits):
| Page No (4 bits) | Offset (4 bits) |

A register called Page Table Base Register (PTBR)

- Holds **starting address of the page table**
- Used by CPU to locate page table in memory

Advantages of Paging

- Paging mainly allows to storage of parts of a single process in a **non-contiguous fashion**.
- With the help of **Paging, the problem** of external fragmentation is solved.
- Paging is one of the simplest algorithms for memory management.

Disadvantages of Paging

- In Paging, sometimes the page table consumes more memory.
- Internal fragmentation is caused by this technique.
- There is an increase in time taken to fetch the instruction since now two memory accesses are required.

Structure of Page Table

- The page table is an array where each entry corresponds to a virtual page.
- Each entry contains:
 - **Frame Number:** The physical frame where the page is stored.
 - **Valid/Invalid Bit:** Indicates if the page is in physical memory.
 - **Protection Bits:** Read/write/execute permissions.
 - **Other bits:** Such as dirty bit (if page has been modified), reference bit (if page was recently accessed).

How Page Table Works

1. CPU generates a virtual address.
2. The virtual address is divided into:
 - **Page Number:** Index into the page table.
 - **Offset:** Location within the page.
3. The page table entry for the page number is accessed.
4. If the valid bit is set, the frame number is combined with the offset to form the physical address.
5. If the valid bit is not set, a page fault occurs, and the OS loads the page from disk.

Page Table Entry Format:

```
+-----+-----+-----+-----+
| Frame Number | Valid Bit | Protection Bits | Other Bits |
+-----+-----+-----+-----+
```


Paging with Translation Look -Aside Buffer(TLBs)

The two memory access problem can be solved by the use of a special fast-lookup hardware cache called associative memory or translation look-aside buffers (TLBs)

Each TLB entry – uniquely identifies each process to provide address-space protection for that process.

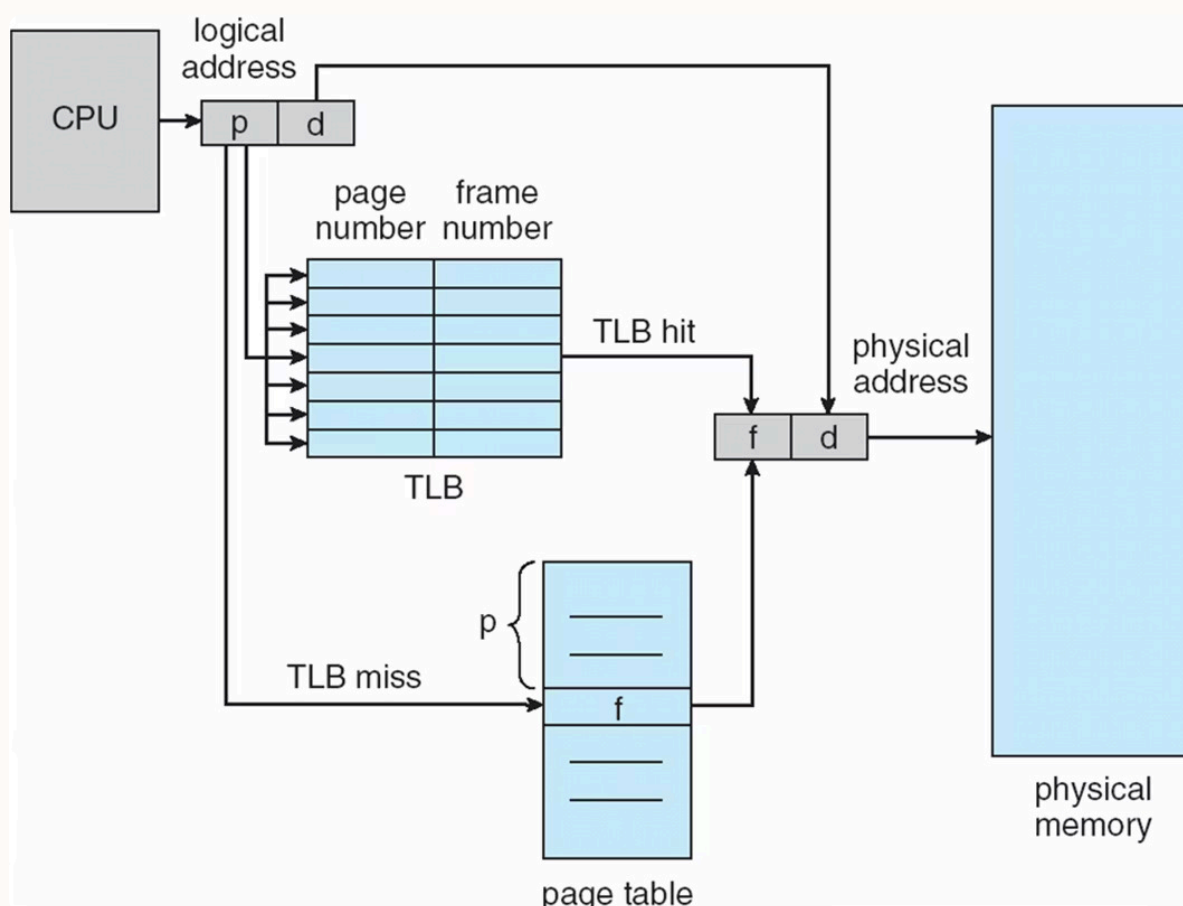
TLBs are typically small size compared to Page Table.

Each time when address translation need to be done, the page reference will be checked in the TLB first.

If it present then it is called TLB Hit, then its frame number will be passed. If it is not present in the table then it called TLB Miss.

On a TLB miss, then page will be searched in the Page Table and later value is loaded into the TLB for faster access next time.

Paging hardware with TLB consists of both TLB table and Page Table.



Virtual Memory

Virtual Memory is a storage mechanism which offers user an illusion of having a very big main memory.

It is done by treating a part of secondary memory as the main memory.

In Virtual memory, the user can store processes with a bigger size than the available main memory.

Instead of loading one long process in the main memory, the OS loads the various parts of more than one process in the main memory.

Virtual memory is mostly implemented with demand paging and demand segmentation

Demand Paging

Demand paging is a memory management technique used by operating systems to optimize the use of memory resources.

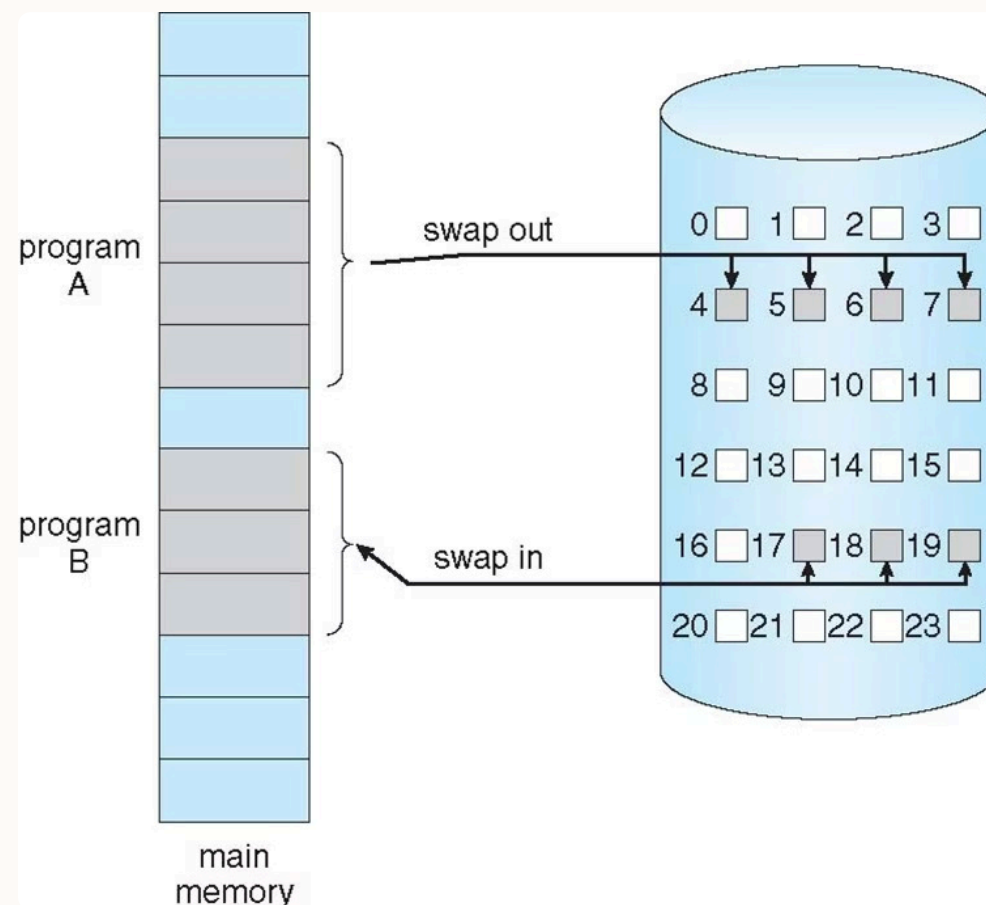
In demand paging, we only load the required pages of a process into the main memory instead of loading the entire process.

A demand paging mechanism is very much similar to a paging system with swapping where processes stored in the secondary memory and pages are loaded only on demand, not in advance.

During the program execution, if the program references a page that may not be available in the main memory then this scheme will bring that page on demand from the secondary storage.

It uses a Lazy swapper – never swaps a page into memory unless page will be needed.

Swapper that deals with pages is called Pager.

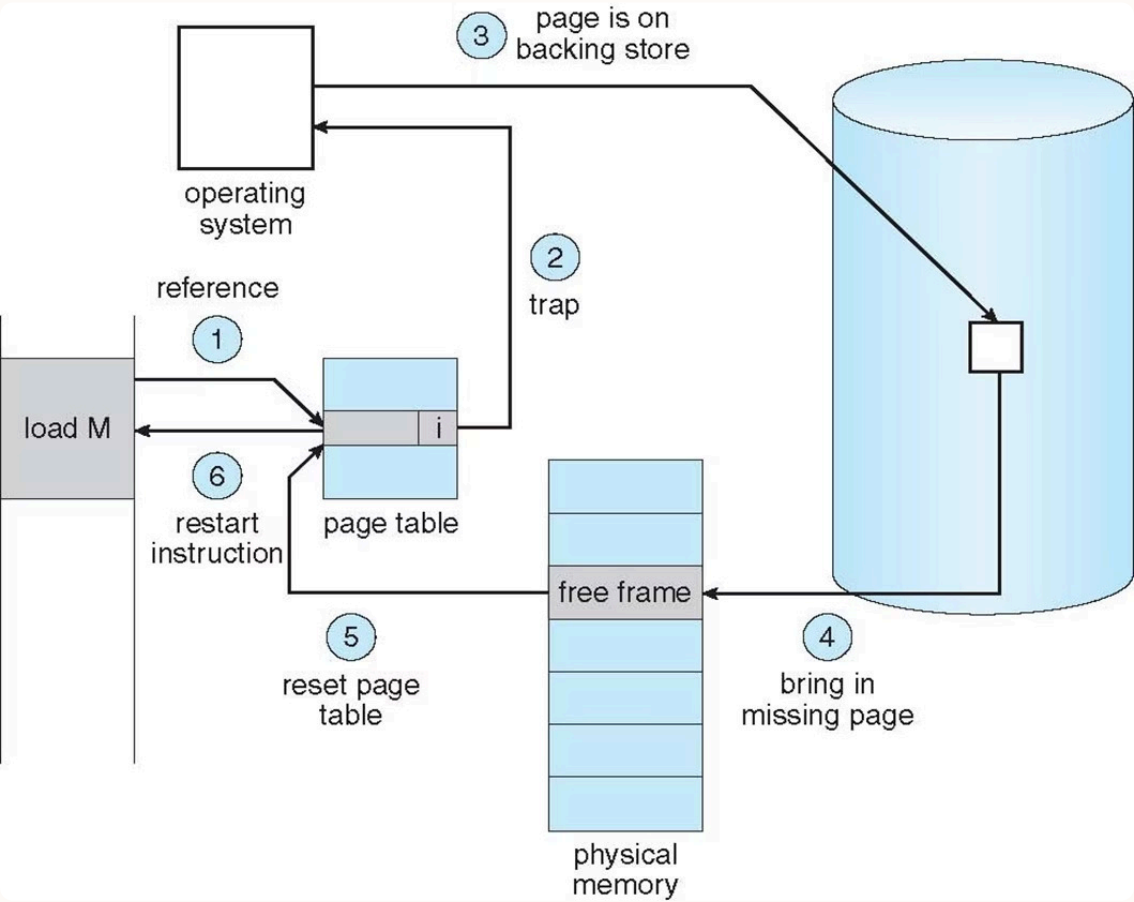


Page Fault

A **page fault** occurs when the CPU tries to access a page that is **not currently in main memory (RAM)**

Steps to follow during page fault:

01	02	03
Reference (CPU request) - CPU executes an instruction like: load M - It checks the page table - Finds that page entry is invalid (i) → page not in memory This is called a Page Fault	Trap to Operating System - Hardware sends a trap (interrupt) to OS - Control goes from user program → Operating System OS now handles the problem	Check in Backing Store (Disk) OS checks whether the required page exists on disk (backing store) If not → program error If yes → continue
04	05	06
Bring Missing Page into Memory - OS finds a free frame in physical memory - Loads the required page from disk → into that frame This step takes high time (disk is slow)	Update Page Table - Page table is updated: - Frame number is filled - Valid bit is set to 1 Now page is available in memory	Step 6: Restart Instruction - The instruction (load M) is executed again This time: - Page is present - Data is fetched successfully



Thrashing

Thrashing is a condition or a situation where the operating system is spending a major portion of its time in servicing the page faults, rather than actual processing.

The effect of Thrashing is:

- CPU becomes idle.

Decreasing the utilization increases the degree of multiprogramming and hence bringing more processes at a time which in fact increases the thrashing exponentially

Page Replacement Algorithm

- Page Replacement Algorithm decides which page to remove, also called swap out when a new page needs to be loaded into the main memory.
- Page Replacement happens when a requested page is not present in the main memory and the available space is not sufficient for allocation to the requested page.
- A page replacement algorithm tries to select which pages should be replaced so as to minimize the total number of page misses.
- If a process requests for page and that page is found in the main memory then it is called page hit.
- If the requested page is not found in the main memory then it is called as page miss or page fault.

Some Page Replacement Algorithms :

1

First In First Out (FIFO)

2

Optimal Page Replacement

3

Least Recently Used (LRU)

First In First Out (FIFO)

This is the simplest page replacement algorithm. In this algorithm, the OS maintains a queue that keeps track of all the pages in memory, with the oldest page at the front and the most recent page at the back.

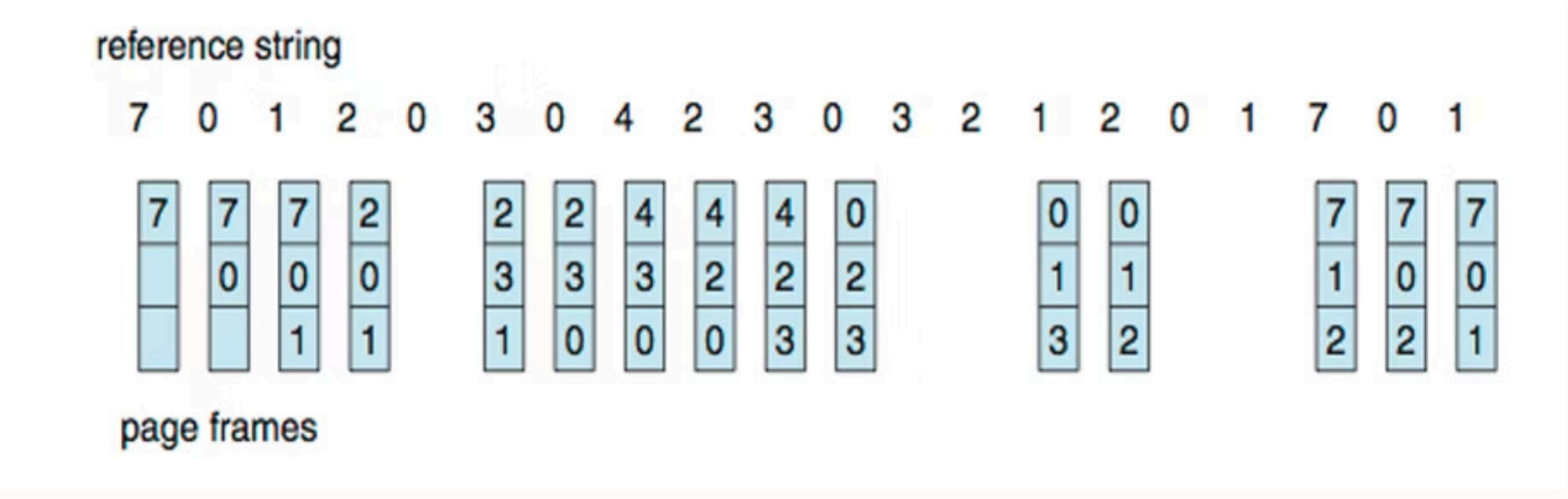
When there is a need for page replacement, the FIFO algorithm, swaps out the page at the front of the queue, that is the page which has been in the memory for the longest time.

- Advantages
- Simple and easy to implement.
 - Low overhead.

- Disadvantages
- Poor performance.
 - Suffers from Belady's Anomaly(i.e. more page faults when we increase the number of page frames).

Example:

In the following example, a reference string is given and there are 3 free frames. There are 20 page requests, which results in 15 page faults



Explanation:

Step 1: Page = 7

Frames: [7 _ _]
✔ Page Fault (empty frame)

Step 2: Page = 0

Frames: [7 0 _]
✔ Page Fault

Step 3: Page = 1

Frames: [7 0 1]
✔ Page Fault (frames full now)

Step 4: Page = 2

- Oldest page = 7 (remove it)

Frames: [2 0 1]
✔ Page Fault

Step 5: Page = 0

Frames: [2 0 1]
✔ Hit (already present)

Step 6: Page = 3

- Oldest page = 0

Frames: [2 3 1]
✔ Page Fault

Step 7: Page = 0

- Oldest page = 1

Frames: [2 3 0]
✔ Page Fault

Step 8: Page = 4

- Oldest page = 2

Frames: [4 3 0]
✔ Page Fault

Step 9: Page = 2

- Oldest page = 3

Frames: [4 2 0]
✔ Page Fault

Step 10: Page = 3

- Oldest page = 0

Frames: [4 2 3]
✔ Page Fault

Step 11: Page = 0

- Oldest page = 4

Frames: [0 2 3]
✔ Page Fault

Step 12: Page = 3

Frames: [0 2 3]
✔ Hit

Step 13: Page = 2

Frames: [0 2 3]
✔ Hit

Step 14: Page = 1

- Oldest page = 2

Frames: [0 1 3]
✔ Page Fault

Step 15: Page = 2

- Oldest page = 3

Frames: [0 1 2]
✔ Page Fault

Step 16:Page = 0

Frames: [0 1 2]
✔ Hit

Step 17: Page = 1

Frames: [0 1 2]
✔ Hit

Step 18: Page = 7

- Oldest page = 0

Frames: [7 1 2]
✔ Page Fault

Step 19: Page = 0

- Oldest page = 1

Frames: [7 0 2]
✔ Page Fault

Step 20: Page = 1

- Oldest page = 2

Frames: [7 0 1]
✔ Page Fault

Final Result

Total Page Faults = 15

Total Hits = 5

Optimal Page Replacement

Optimal page replacement algorithm is mainly to solve the problem of Belady's Anomaly

Optimal page replacement algorithm has the lowest page fault rate of all algorithms

An optimal page replacement algorithm exists and has been called OPT. The working is simple “Replace the page that will not be used for the longest period of time”

Example:

consider the following reference string

- The first three references cause faults that fill the three empty frames.
- The references to page 2 replaces page 7, because 7 will not be used until reference 18.
- The page 0 will be used at 5 and page 1 at 14.
- With only 9 page faults, optimal replacement is much better than a FIFO, which had 15 faults.
- This algorithm is difficult to implement because it requires future knowledge of reference strings.
- Replace page that will not be used for longest period of time

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1



page frames

Least Recently Used (LRU) Algorithm

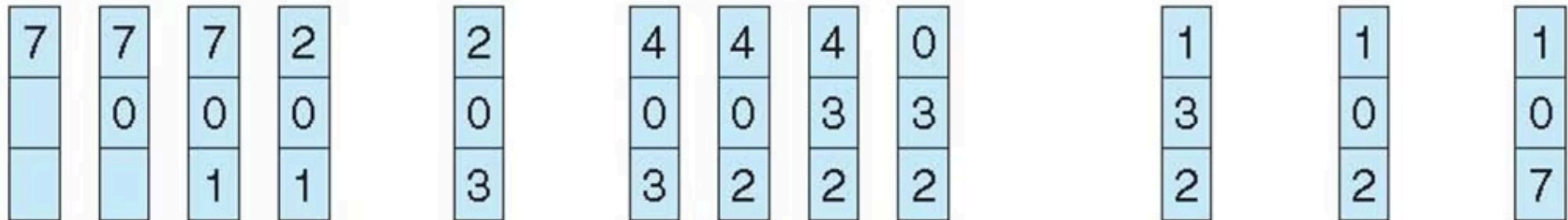
Use past knowledge rather than future

Replace page that has not been used in the most amount of time

Associate time of last use with each page.

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1



page frames

- 12 faults – better than FIFO but worse than OPT